

# Vibe Coding Lab — Chat History

Arm Wrestle Showdown project • GitHub Copilot / VS Code Chat

This PDF contains the user prompts and the corresponding final assistant responses from the exported Vibe Coding session. Internal tool calls, diagnostic traces, and hidden reasoning are omitted for readability.

## Conversation 1: Initial Project Analysis

### User Prompt

I am doing a Vibe Coding lab on this project.

First, inspect the README.md and all relevant source files.

Do NOT modify any files yet.

The original game currently has a problem: when I run python main.py, the arm moves automatically and the computer wins even when I press the Left and Right arrow keys.

Analyze the project and identify:

1. What is causing the current broken behavior.
2. Where player keyboard input is handled.
3. Where arm\_position is updated.
4. How the AI movement works.
5. How stamina works.
6. What changes/features are required by the README.

Do not make any changes yet. Just explain your findings and identify the exact files and functions that need modification.

### GitHub Copilot Response

No files were changed.

1. **Cause of the broken behavior:** In game/game\_engine.py, GameEngine.handle\_event() adds `4.2` to arm\_position for both Left and Right key presses. Positive positions move toward the computer's win threshold, so player input helps the computer. Meanwhile, GameEngine.update() also adds a positive AI force every frame. The two forces therefore push in the same direction, and the computer wins even when the player presses the arrows.

2. **Keyboard input:** main() in main.py forwards each non-quit event to engine.handle\_event(event). GameEngine.handle\_event() processes KEYDOWN events while the game is playing. It accepts alternating Left/Right presses, ignores a repeated press of the same key, and blocks input when stamina is at or below 10.

3. **Arm position updates:** GameEngine.handle\_event() changes arm\_position by `+4.2` on each accepted player press; this is the inverted operation described in README Task 1. GameEngine.update() adds the AI force every playing frame and checks whether arm\_position has reached either win threshold.

4. **AI movement:** In GameEngine.update(), the AI picks a random multiplier between `0.3` and `1.0` each frame, then adds `ai\_strength \* ai\_variance` to arm\_position. The average push is positive and constant in character: there is no buildup, surge, or exhausted phase yet.

5. **Stamina:** An accepted alternating key press costs `2.0` stamina, clamped at zero. Each playing-frame update restores `0.8` stamina, up to the maximum of `100.0`. Inputs stop at `stamina <= 10`; the bar turns red at `stamina <=

25`, but there is no specific exhaustion warning. The regeneration is frame-based, so at the 60 FPS target it can restore up to 48 stamina per second.

6. **README requirements and files to modify:**

- **Task 1:** Fix the player push direction in `GameEngine.handle_event()`.
- **Task 2:** Add AI energy/surge and recovery behavior in `GameEngine.update()`.
- **Task 3 and Task 4:** Both request the same exhaustion warning. Add the visible warning in `GameEngine.render()`. The duplicate task doesn't appear to require a second distinct feature.
- **Rematches:** If the AI feature adds state such as energy or surge timers, reset it in `GameEngine.reset()` along with the existing arm, stamina, key, winner, and game state.
- **Submission checklist:** The README also asks for before/after gameplay videos and the complete LLM chat-history link; these are deliverables, not code changes.

appears to be wiring the game loop correctly and does not need modification for the described behavior.

## Conversation 2: Task 1 — Fix Inverted Arm Push

### User Prompt

Now implement Task 1 from the README.

Fix the player's arm movement direction in `GameEngine.handle_event()`.

Currently, when an accepted alternating Left/Right key press occurs, the code adds +4.2 to `arm_position`.

This is incorrect because positive `arm_position` moves toward the computer's win threshold.

Change the player input so that accepted Left/Right presses move `arm_position` toward the player's side, while preserving all the existing rules:

- alternating Left/Right input
- ignoring repeated presses of the same key
- stamina cost
- input being blocked when `stamina <= 10`
- existing win conditions

Make only the changes necessary for Task 1.

Do not modify the AI behavior, stamina system, UI, or other features yet.

After making the change, explain exactly what you changed.

### GitHub Copilot Response

Updated `game/game_engine.py`: accepted Left and Right presses now subtract `4.2` from `arm_position`, moving it toward the player's negative win threshold. Alternation, repeated-key filtering, stamina cost and cutoff, AI behavior, and win conditions are unchanged.

**\*\*Validation:\*\*** Handler-level assertions passed for movement direction, alternation, repeated-key rejection, stamina cost, and input blocking at 10 stamina. The configured Python interpreter lacked Pygame, so the check used a small in-memory Pygame stub.

## Conversation 3: Task 2 — AI Energy / Surge / Recovery

### User Prompt

Now implement Task 2 from the README.

Add the AI energy/surge and recovery behavior in `GameEngine.update()`.

Requirements:

- Keep the existing normal AI behavior.
- Add an AI energy/stamina system that gradually builds while the AI is in its normal state.
- When enough energy is accumulated, trigger a power surge.
- During the surge, the AI should apply noticeably stronger force for approximately 1–2 seconds.
- After the surge ends, put the AI into an exhausted/recovery state.
- During AI exhaustion, reduce the AI's resistance/force so the player has an opportunity to push back.
- Allow the AI to recover and eventually return to its normal state.
- Make the state transitions repeatable during a match.
- Reset all new AI energy/surge/recovery state in `GameEngine.reset()` when a rematch starts.

Preserve the existing player input, player stamina, arm movement, win conditions, and rendering unless a small change is required to display the new AI state.

Do not implement the exhaustion warning yet. That is Task 3.  
Make the smallest clean changes necessary.

After implementing it, explain exactly what states, timers, and force changes you added.

### GitHub Copilot Response

Implemented Task 2 in `game/game_engine.py`. The AI now cycles through three states:

- **NORMAL:** Keeps the original force calculation unchanged while energy builds for 5 seconds.
- **SURGE:** Applies 2.5 times the normal force for approximately 1.5 seconds.
- **RECOVERY:** Applies 0.2 times the normal force for 3 seconds, giving the player a chance to push back. The AI then returns to NORMAL and begins building energy again.

The simulation timing assumes the game's existing 60 FPS update loop. `reset()` clears the AI energy and timer and returns its state to NORMAL. Player input, player stamina, win conditions, and rendering were not changed.

**Validation:** Deterministic checks passed for phase transitions, repeat cycling, force scaling, and reset. The file diagnostic check reported no errors.

## Conversation 4: Task 2 — Testing / Behavior Analysis

### User Prompt

I tested Task 2 after your implementation.

Observed behavior:

- If I press the player keys quickly, the player wins.
- If I leave more time between presses, the computer wins.
- I cannot visibly tell when the AI enters its surge state or recovery/exhaustion state.

Please inspect the current implementation of Task 2 again.

Do not modify the code yet.

Explain:

1. Exactly how long it takes for the AI to build enough energy for a surge.
2. Exactly how long the surge lasts.
3. How much stronger the AI becomes during the surge.
4. How the AI's force changes during exhaustion/recovery.
5. What conditions cause the AI to transition between NORMAL, SURGE, and recovery/exhausted states.
6. Whether these changes are actually large enough to be noticeable during normal gameplay.

Also identify the exact variables and lines/functions controlling these values.

Do not make any changes yet.

### GitHub Copilot Response

No code changes made.

1. **Energy buildup:** `ai_energy` gains `1/60` on each `update()` while the AI is NORMAL. Because repeated floating-point additions remain slightly below `5.0` after 300 updates, the threshold is reached on update 301: about **5.017 seconds** at 60 updates per second. targets 60 FPS, but that is a cap, so actual wall-clock time can be longer if the game runs below 60 FPS. See `game/game_engine.py#L22`, the `game/game_engine.py#L58`, and the `game/game_engine.py#L59`.
2. **Surge duration:** The configured threshold is **1.5 seconds**. The timer's repeated `1/60` additions cross it on the 91st timer update, so the SURGE state lasts about **1.517 seconds** at 60 updates per second. The surge force begins on the same update that NORMAL crosses the energy threshold. See the `game/game_engine.py#L62`.
3. **Surge force:** The multiplier is **2.5× normal force**, or 150% more than normal. Since the random variance still applies, force per update ranges from `0.2625` to `0.875` during surge, versus `0.105` to `0.35` normally. See the `game/game_engine.py#L74`.
4. **Recovery force:** RECOVERY uses **0.2× normal force**, an 80% reduction. Its configured timer is 3 seconds; floating-point accumulation crosses that threshold on the 181st timer update, about **3.017 seconds** at 60 updates per second. When recovery ends, the code returns to NORMAL and resets energy to zero. See the `game/game_engine.py#L68` and the `game/game_engine.py#L74`.
5. **State transitions:** NORMAL accumulates energy until it is at least `5.0`, then switches to SURGE. SURGE accumulates `ai_state_timer` until it is at least `1.5`, then switches to RECOVERY. The recovery branch accumulates the timer until it is at least `3.0`, then returns to NORMAL and clears energy. The new state and timer start in `game/game_engine.py#L21` and are cleared for a rematch in `game/game_engine.py#L94`.
6. **Noticeability:** The force changes are large enough to affect arm movement, especially the 2.5× surge, so players may notice the arm moving faster or slower. But there is no explicit indicator: `game/game_engine.py#L99` does not

display ai\_state. The arm position is the only visual signal, and it also reflects player input and randomized AI force. That makes the phase change easy to miss, even though the force changes are substantial.

These values explain the reported outcome: AI force is applied every update, while player force is applied only on accepted key presses. Longer gaps between presses give the ongoing AI force more time to move the arm toward the computer's threshold.

## Conversation 5: Task 3 — Exhaustion Warning

### User Prompt

Now implement Task 3 from the README.

Add a clear visual exhaustion warning for the player.

Current behavior:

- Player stamina decreases when accepted alternating key presses are made.
- Player input becomes blocked when stamina  $\leq 10$ .
- The stamina bar already changes color when stamina is low.

Add a visible "EXHAUSTED!" warning near the player's stamina bar whenever player stamina  $\leq 10$ .

Requirements:

- The warning should be clearly visible.
- Make it flash or otherwise visually stand out.
- It should only appear while the player is exhausted.
- It should disappear automatically when stamina recovers above 10.
- Do not change the existing stamina mechanics or input behavior.
- Do not change the AI surge/recovery system.
- Implement the warning in `GameEngine.render()` as described in the README.
- Keep the change small and consistent with the existing Pygame UI.

Task 3 and Task 4 in the README appear to request the same exhaustion warning, so implement this feature only once.

After implementing it, explain exactly what you changed and how the warning is triggered.

### GitHub Copilot Response

Added a flashing red `EXHAUSTED!` label beside the stamina bar in `game/game_engine.py`. It appears only when `stamina  $\leq 10$ `, matching the existing input cutoff, and is drawn on alternating 300 ms intervals. It disappears automatically above 10 stamina. Stamina mechanics, input handling, and the AI system are unchanged.

**\*\*Validation:\*\*** The render check passed for the threshold and flash phases; no file diagnostics were reported.

## Conversation 6: Task 3 — Stamina Regeneration Fix

### User Prompt

I tested Task 3 and the exhaustion warning is not appearing.

Please inspect the current stamina implementation and fix the issue.

The current game regenerates 0.8 stamina on every update frame while the game runs at 60 FPS, while each accepted player key press only costs 2 stamina. This makes it extremely difficult for normal rapid alternating input to ever reach the  $\leq 10$  exhaustion threshold.

I need Task 3 to be realistically demonstrable during normal gameplay.

Requirements:

1. Keep the maximum stamina at 100.
2. Keep the exhaustion threshold at 10.
3. Keep the existing 2.0 stamina cost per accepted alternating key press.
4. Adjust the stamina regeneration so it is balanced and occurs at a reasonable per-second rate rather than effectively restoring 48 stamina per second at 60 FPS.
5. Do not remove the existing stamina system.
6. Keep the input blocking behavior when stamina  $\leq 10$ .
7. Keep the "EXHAUSTED!" visual warning from Task 3.
8. Make sure stamina can actually reach  $\leq 10$  when the player rapidly alternates Left and Right for a reasonable period.
9. Make sure stamina naturally recovers after the player stops pressing.
10. Do not modify the AI surge/recovery system or player arm movement.

Use the smallest clean change necessary.

After making the change, explain the old regeneration rate, the new regeneration rate, and approximately how long rapid pressing should take to reach the exhaustion threshold.

### GitHub Copilot Response

Updated stamina regeneration in game/game\_engine.py. It now restores **1.5 stamina per second** at the existing 60 FPS update step, down from **48 per second** (0.8 per frame). The maximum of 100, 2-point press cost,  $\leq 10$  input block, and exhaustion warning are unchanged. AI behavior and arm movement were not modified.

At about six accepted alternating presses per second, the player spends 12 stamina per second and regains 1.5, for a net loss of 10.5 per second. That should reach the cutoff in roughly **8.6 seconds** from full stamina. A deterministic simulation reached it at **8.5 seconds**, before the match ended. Stamina continues to regenerate when pressing stops, up to its existing maximum.

The focused stamina and recovery checks passed, and the file diagnostic check reported no errors.